



Ben-Gurion University of the Negev
Faculty of Engineering Sciences
Department of Software and Information systems

Deep Reinforcement Learning

Assignment #2

By

Oz Elbaz – 204388763
Ido Gurevich - 205478068

Section 1 - Monte-Carlo Policy Gradient (REINFORCE):

Theoretical questions

"What does the value of the advantage estimate reflect ?

Why is it better to follow the gradient computed with the advantage estimate instead of just the return itself?"

The advantage value represents the difference between the actual future reward obtained and the estimated value function of a state. A positive advantage means the action taken in state resulted in a higher return than the expected value under the policy, indicating better-than-average performance. Conversely, a negative advantage indicates the action performed worse than the policy's average expectation for that state. In addition, using the advantage estimate instead of the raw return in gradient computation reduces variance and focuses updates on meaningful differences in performance, enabling faster and more stable policy convergence.

"The reduction of the baseline does not introduce bias to the expectation of the gradient since ?.

$$E_{\pi_{\theta}}[\nabla \log \pi_{\theta}(a_t, s_t) b(s_t)] = 0$$

What is the prerequisite condition for that equation to be true? Prove the equation.

The prerequisite condition for the equation to hold is that the baseline depends only on the state and is independent of the actions.

$$\begin{aligned} (*) \nabla \log \pi_{\theta}(a_t, s_t) &= \frac{\nabla \pi_{\theta}(a_t, s_t)}{\pi_{\theta}(a_t, s_t)} \\ (**) \sum_{a_t} \pi_{\theta}(a_t, s_t) &= 1 \rightarrow \sum_{a_t} \nabla \pi_{\theta}(a_t, s_t) = 0 \\ E_{\pi_{\theta}}[\nabla \log \pi_{\theta}(a_t, s_t) b(s_t)] &= \sum_{a_t} \pi_{\theta}(a_t, s_t) \nabla \log \pi_{\theta}(a_t, s_t) b(s_t) \stackrel{(*)}{\Rightarrow} \\ \sum_{a_t} \pi_{\theta}(a_t, s_t) \frac{\nabla \pi_{\theta}(a_t, s_t)}{\pi_{\theta}(a_t, s_t)} b(s_t) &= \sum_{a_t} \nabla \pi_{\theta}(a_t, s_t) b(s_t) = b(s_t) \sum_{a_t} \nabla \pi_{\theta}(a_t, s_t) \stackrel{(**)}{\Rightarrow} b(s_t) \cdot 0 \\ &= 0 \end{aligned}$$

Implementation of reinforce and reinforce with baseline:

In this section, we implemented and compared two algorithms: REINFORCE and REINFORCE with Baseline, using the CartPole environment from Gymnasium. The environment's state is represented by a $4 \times 14 \times 1$ vector, and the action space consists of two actions: turning left or right.

Architecture:

- **Reinforce:**
Used a single neural network for the policy, with hidden layer sizes of [16,8].
- **Reinforce with baseline:**
Used two neural networks: one for the policy and one for the value function, both with hidden layer sizes of [16,8].
- **Hyperparameters trials:**
Learning rate policy net/ value net : 0.001\0.005\0.01.
Discount factor: 0.999\0.99\0.995.
Max number of episodes: 1000.
Max steps per episode: 500.

Results:

To identify the best hyperparameters, we tested various combinations and analyzed their performance. As shown in Figure 1, we provide two examples of hyperparameter tuning trials for each algorithm. The upper panel illustrates the cumulative reward per episode for a learning rate of 0.001, while the lower panel corresponds to a learning rate of 0.005.

For the learning rate of 0.001 (upper panel), the REINFORCE algorithm exhibited significant oscillations in the cumulative reward per episode, whereas the REINFORCE with Baseline algorithm showed a smoother and more stable learning curve. When using a learning rate of

0.005 (lower panel), both algorithms demonstrated improved learning curves compared to the results with 0.001, indicating faster convergence and better performance.

Overall, the REINFORCE with Baseline algorithm outperformed the REINFORCE algorithm in terms of stability and cumulative rewards. The algorithm converged at around 600 episodes with the following hyperparameters:

- Learning rate policy net/ value net : 0.005
- Discount factor: 0.999.
- Max number of episodes: 1000.
- Max steps per episode: 500.

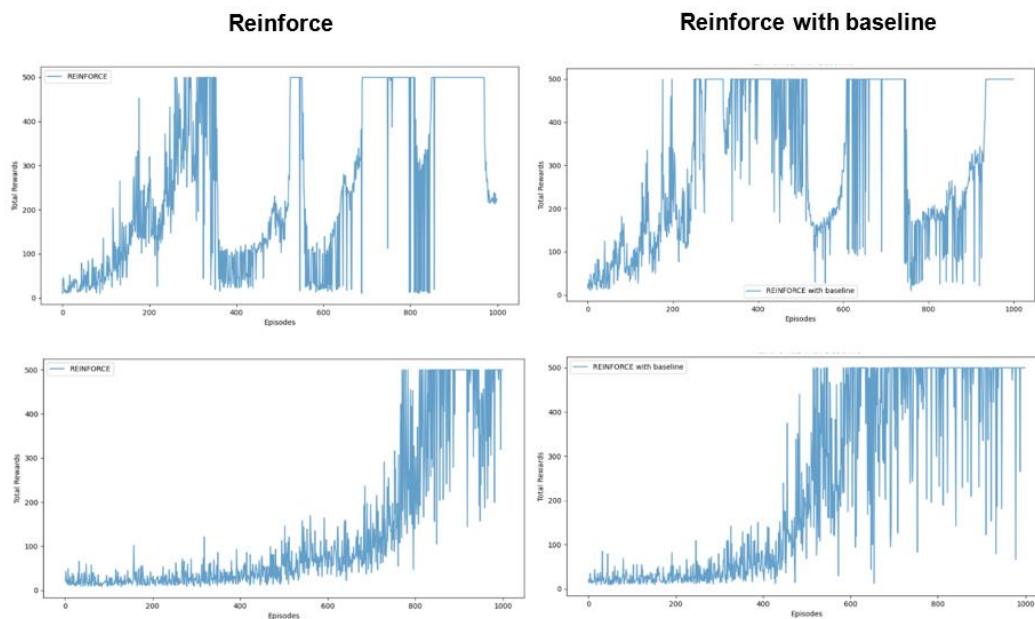


Figure 1: Hyperparameters tuning. The left panel represents the cumulative reward of the reinforce algorithm and the right panel is the cumulative reward per episode of the reinforce with baseline algorithm. Each row of graph is corresponds to a different learning rate.

To sum up, these results highlight the benefits of using a baseline, which helps reduce variance and improves learning stability, leading to more consistent and reliable policy updates.

Section 2 – Advantage actor critic:

Theoretical questions

"Why is using the TD-error of the value function for the update of the policy network parameters is practically the same as using the advantage estimate (of equation 1)?"

Proof:

$$\begin{aligned}\delta_{\pi_\theta} &= R_{t+1} + \gamma \cdot V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t) \rightarrow \\ E_{\pi_\theta}[\delta_{\pi_\theta} | s, a] &= E[R_{t+1} + \gamma \cdot V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t) | s, a] = E[R_{t+1} + \gamma \cdot V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t)] \\ &= E[R_{t+1} + \gamma \cdot V_{\pi_\theta}(s_{t+1})] - V_{\pi_\theta}(s_t) = Q_{\pi_\theta} - V_{\pi_\theta}(s_t) = \hat{A}(s, a)\end{aligned}$$

Updating the policy network parameters using the TD error of the value function is equivalent to using the advantage estimate.

"Explain which function is the *actor* and which is the *critic* of the model and what is the role of each one."

The policy represents the actor, while the value function represents the critic. The critic's role is to estimate the value function, providing feedback to the actor on the quality of its actions and how to adjust them. This feedback can be based on either the action-value (Q function) or the state-value (value function). The actor determines which action to take based on the policy distribution and is updated according to the directions provided by the critic, such as through policy gradients.

Implementation of advantaged actor-critic:

In this section, we implemented the advantaged actor critic algorithm using the same environment as in section 1. To identify the best hyperparameters, we tested various architectures and combinations of hyperparameters and analyzed their performance. Here we presents five different architectures with 5 different combinations of hyperparameters:

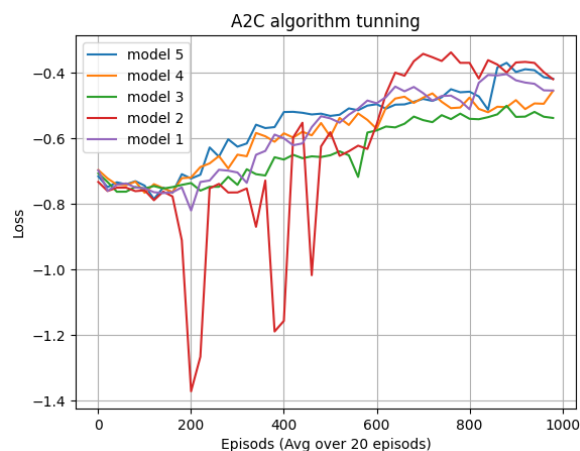
Models:

Model/ parameters	PolicyNet hidden sizes	Value Net hidden_sizes	LR policyNet	LR_ValueNet	Discount factor
(1)	[256, 128, 32]	[256, 128, 32]	0.0009	0.0009	0.999
(2)	[64, 64, 32]	[64, 64, 32]	0.005	0.005	0.995
(3)	[128, 64, 16]	[128, 64, 16]	0.0009	0.0009	0.999
(4)	[64, 64, 32]	[64, 64, 32]	0.0009	0.0009	0.999
(5)	[256, 64, 32]	[256, 64, 32]	0.001	0.001	0.999.

Table 1: Architectures and hyperparameters for each trial.

Results:

We experimented with several combinations of architectures to identify the best-performing algorithm. As shown in Figure 2, the results of four different attempts are presented. For start, all models were able to converge at some level, where model (3) got the worst results in terms of loss, model (2) did fine in the end but was unstable throughout the run, and models (1), (4) and (5) got similar results. Between those three, it seems like model (5) has the best results.



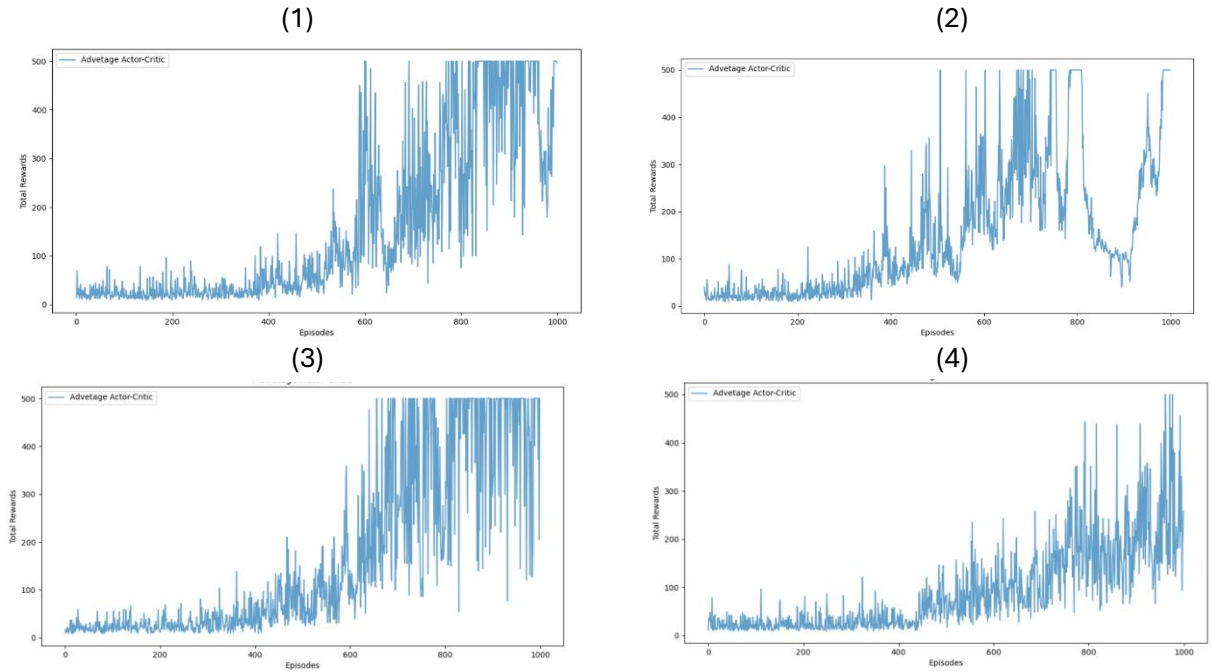


Figure 2: Results of Advantage Actor-Critic: Each graph corresponds to the respective model listed in Table 1..

Figure 3 highlights the results of model 5, which outperformed all other models. The cumulative reward reached its peak in just over 600 episodes. The losses increased over time due to the use of gradient ascent in this algorithm, which inherently drives the optimization in the direction of maximizing rewards.

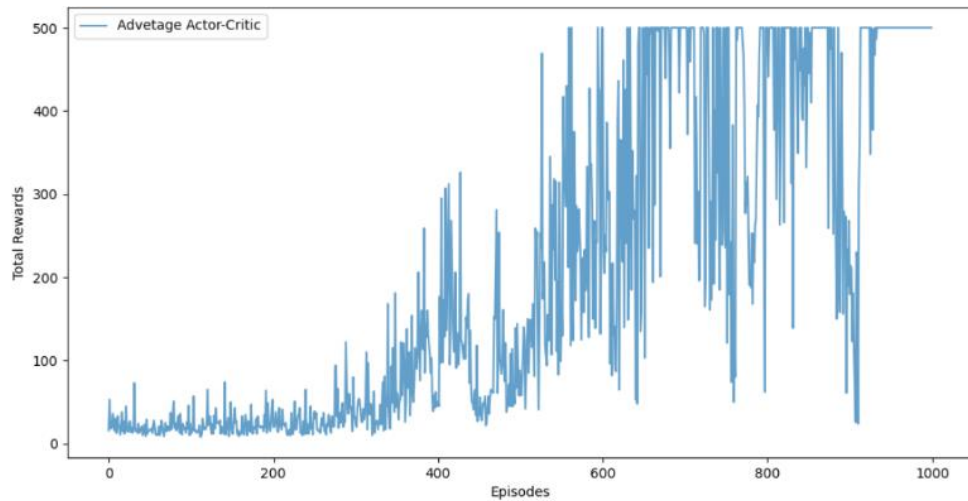


Figure 3: Results of model 5: the summed reward per episode.

Comparison between A2C and REINFORCE with Baseline

In Figure 4 below, we present the rewards over episode comparison between the two algorithms. The overall rewards accumulate by the REINFORCE algorithm, seem to be greater. Although, we've been expecting the opposite. This can be explained by the mis-tuning of the hyper-parameters and architecture. Another explanation can be due to the fact that the REINFORCE algorithm iterates at each step, while the A2C needs to iterate at every episode. Making the REINFORCE less stable, but learns the task faster (needs fewer episodes) compare to the A2C.

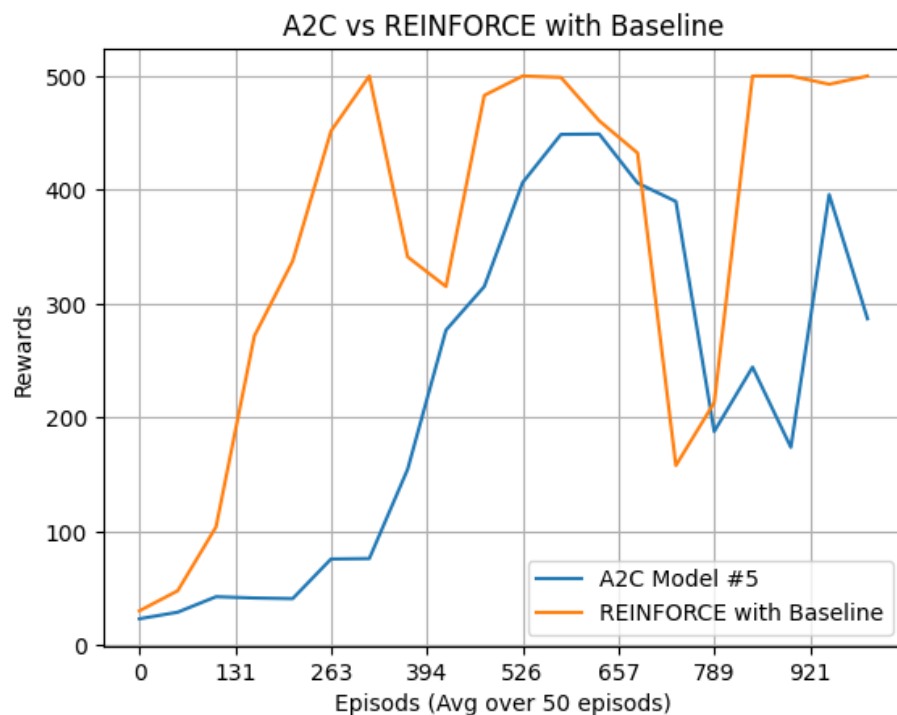


Figure 4: A2C vs REINFORCE with Baseline

Running the scripts:

All the scripts for this assignment can be found either in the ZIP file that uploaded to the moodle, or in the following Git repo (branch: "Assignment#2"):
https://github.com/oz182/DRL_Assignments.git

